

Optimization of the MiniAD Library for Convolutional Neural Networks in Julia

Katarzyna Dudek, Izabela Kolodziejska
Warsaw University of Technology
Faculty of Electrical Engineering

Abstract—This paper presents the optimization process of the MiniAD library written in Julia. MiniAD implements automatic differentiation using backward accumulation and a computational graph. The library supports training a convolutional neural network on the FashionMNIST dataset. The goal of the KM2 project was to improve execution time, reduce unnecessary memory allocations, and increase computational efficiency without reducing classification accuracy. The largest performance improvement was achieved by re-designing the Conv2D layer using the im2col transformation and GEMM operations from the BLAS library. Additional work focused on buffer reuse, parametric typing of CNN operators, profiling, and rejecting ineffective micro-optimizations. The final optimized Julia implementation reduced the full 3-epoch training time from 67.91 s to 28.75 s while maintaining high test accuracy of 87.59%. The optimized implementation was also compared with reference PyTorch and Flux implementations.

Index Terms—automatic differentiation, Julia, CNN, BLAS, im2col, optimization, FashionMNIST

I. INTRODUCTION

In the KM1 project, the MiniAD library was implemented as a custom automatic differentiation library based on backward accumulation and a computational graph. The library was extended with the operations required for training a convolutional neural network:

- Conv2D,
- MaxPool2D,
- Dropout,
- SoftMax.

The network was trained on the FashionMNIST dataset for 3 epochs using stochastic gradient descent. The baseline implementation satisfied the accuracy requirement, but its execution time and memory behavior left room for optimization.

The main goal of the KM2 project was to optimize the library in terms of:

- training time,
- memory management,
- computational efficiency,
- stability of the implementation.

The focus of this paper is only on the optimization process and its verification. The automatic differentiation mechanism itself, the network architecture, and the training setup are treated as fixed project assumptions.

All final experiments used the same model architecture, FashionMNIST dataset, batch size, optimizer, learning rate, number of epochs, and Dropout probability.

Each optimization was verified using:

- unit tests,
- short benchmarks on 100 or 200 batches,
- full 3-epoch training with test accuracy measurement,
- memory allocation measurements.

II. NETWORK ARCHITECTURE

All experiments used the same CNN architecture: Conv2D(1–6, 3x3, pad=1) – MaxPool(2x2) – Conv2D(6–16, 3x3, pad=1) – MaxPool(2x2) – Flatten – Dense(784–84) – ReLU – Dropout(0.4) – Dense(84–10).

The model was trained for 3 epochs with SGD, learning rate 0.01, and batch size 10.

III. BASELINE IMPLEMENTATION

The KM1 baseline implementation used a direct Conv2D implementation based on multiple nested loops. For every output element, the implementation iterated over:

- output channels,
- input channels,
- kernel height,
- kernel width,
- spatial positions,
- batch elements.

The backward pass was also implemented directly using nested loops over tensors. This approach was simple and correct, but it did not efficiently use optimized numerical kernels.

The baseline result is shown in Table I.

TABLE I
KM1 BASELINE RESULTS

Metric	Result
Full 3-epoch training time	67.91 s
Final test accuracy	87.13%

The baseline accuracy satisfied the project requirement, but the execution time was significantly higher than the optimized KM2 version.

IV. OPTIMIZATION METHODOLOGY

The optimization process followed a conservative rule: only changes that improved performance without reducing accuracy or breaking tests were kept.

The main measurements were:

- full training time,
- final test accuracy,
- execution time of selected training stages,
- memory allocations during training and evaluation,
- correctness through unit tests.

Several experiments were rejected because they increased execution time or memory usage despite being logically correct. This was important because many low-level optimizations did not target the actual bottleneck.

The final optimization path consisted of:

- 1) buffer preallocation,
- 2) branchless direct Conv2D paths,
- 3) Conv2D redesign using im2col and GEMM,
- 4) Dropout mask buffer reuse,
- 5) parametric typing of CNN operators,
- 6) explicit use of BLAS GEMM.

V. BUFFER PREALLOCATION

The first optimization focused on preallocating reusable buffers in Conv2D and MaxPool2D layers.

In the baseline implementation, some temporary arrays were repeatedly allocated for consecutive batches. This increased memory allocations and garbage collector pressure.

The following changes were introduced:

- reusable Conv2D buffers,
- reusable MaxPool2D buffers,
- reduced repeated allocation of intermediate tensors.

The results are shown in Table II.

TABLE II
EFFECT OF BUFFER PREALLOCATION

Version	Time	Accuracy
KM1 baseline	67.91 s	87.13%
Preallocation	64.20 s	87.13%

This reduced full training time by approximately 5.5% without changing test accuracy.

VI. CONV2D USING IM2COL AND GEMM

The largest optimization was achieved by redesigning Conv2D.

The direct convolution implementation performed many small operations in Julia loops. Although correct, this approach did not efficiently use optimized numerical routines. The optimized version replaced direct convolution with the following pipeline:

- 1) im2col,
- 2) GEMM,

3) col2im in the backward pass.

where:

- W has shape $C_{out} \times K$,
- X_{col} has shape $K \times P$,
- Y has shape $C_{out} \times P$,
- K is the flattened kernel size multiplied by the number of input channels,
- P is the number of extracted spatial patches.

The backward pass uses:

$$dW = dY \cdot X_{col}^T$$

and:

$$dX_{col} = W^T \cdot dY$$

Finally, the col2im operation accumulates dX_{col} back into the input gradient tensor.

The main benefit is that many small tensor operations are replaced by matrix multiplications executed by BLAS. This substantially improves CPU performance.

TABLE III
EFFECT OF IM2COL + GEMM CONV2D

Version	Time	Accuracy
Branchless direct Conv2D	63.83 s	87.13%
im2col + GEMM	28.66 s	87.55%

This optimization reduced training time by more than a factor of two. It was the most important performance improvement in the whole KM2 project.

VII. PARAMETRIC TYPING OF CNN OPERATORS

Another important part of the final KM2 implementation was parametric typing of CNN operator structures.

Earlier versions used more general cache fields for some internal buffers. This made the implementation more flexible, but it provided less type information to the Julia compiler.

The final implementation introduced parametric operator types:

$$Conv2D\text{Op}\{T\}$$

and:

$$MaxPool2D\text{Op}\{T\}$$

where:

$$T <: AbstractFloat$$

The main changes were:

- Conv2D buffers were typed as arrays or matrices of element type T ,
- MaxPool2D output and gradient buffers were typed as arrays of element type T ,
- the Conv2D backward buffer dx_{col} was typed as a matrix,
- constructors explicitly passed the element type T ,
- the element type was inferred from the computational graph rather than from uninitialized output fields.

TABLE IV
EFFECT OF PARAMETRIC TYPING

Version	Tests	Time	Accuracy
Before parametric typing	46/46	29.76 s	87.59%
Parametric CNN operators	46/46	29.68 s	87.59%

The full training time changed only slightly, but memory allocations in training were reduced. This is discussed in the memory section.

The conclusion is that parametric typing was not the main time optimization, but it improved type stability, reduced allocations, and made the implementation closer to idiomatic Julia.

VIII. TIME PROFILING

To identify the dominant execution costs, an additional profiling script was implemented:

```
benchmark_profile.jl
```

The script measured:

- forward propagation,
- backward propagation,
- SGD parameter update,
- evaluation.

Representative results for 200 training batches are shown in Table V.

TABLE V
TIME PROFILING FOR 200 TRAINING BATCHES

Stage	Time [s]
Forward	1.48
Backward	1.05
SGD step	0.17
Evaluation	0.09

The results show that forward and backward propagation dominate the total execution time. This confirms that optimizing Conv2D was the correct main target.

The parameter update step is much cheaper than forward and backward propagation. Therefore, further optimization of SGD alone would not provide a comparable gain.

IX. MEMORY ALLOCATIONS

Memory allocations were measured using:

```
@allocated
```

The measurement covered:

- training over 100 batches,
- evaluation over 100 batches.

The results are shown in Table VI.

The initial im2col version increased training allocations because it constructs explicit column matrices from convolution windows. This is a known trade-off of im2col: faster

TABLE VI
MEMORY ALLOCATIONS FOR 100 BATCHES

Version	Train	Eval
KM1 baseline	921.76 MB	60.53 MB
KM2 before final typing	970.11 MB	5.65 MB
Final KM2	884.92 MB	5.57 MB

matrix multiplication is obtained at the cost of additional memory.

However, the final KM2 version reduced training allocations below the KM1 baseline. The main reasons were:

- buffer reuse,
- improved type stability,
- parametric typing of CNN operator caches.

Evaluation allocations were reduced from 60.53 MB to 5.57 MB. This is a reduction of approximately:

90.8%

The evaluation path benefits strongly from reusable buffers because no backward pass is required.

X. REJECTED OPTIMIZATION EXPERIMENTS

Several optimization attempts were tested but rejected because they did not improve the final performance.

Examples include:

- copying batches instead of using views,
- stricter cache typing without proper parametric design,
- removing `copy(g)` in gradient accumulation,
- reusing padded input in Conv2D backward,
- caching maximum indices in MaxPool,
- `@views` in Conv2D backward,
- `@simd` in selected loops,
- `@inline` on helper functions,
- additional `@fastmath` annotations in selected kernels,
- branch hoisting in the im2col path.

XI. STATICARRAYS EXPERIMENT IN MAXPOOL

One rejected experiment was inspired by the idea that small fixed-size arrays can sometimes be faster when allocated statically.

Since MaxPool uses small 2×2 windows, an experimental fast path was implemented using static small arrays. The idea was to represent each pooling window as a small fixed-size object and compute the maximum using this representation.

The experiment passed all unit tests:

46/46

However, performance results were unstable. Short benchmarks showed high variance, and memory allocation measurements increased during training.

The final conclusion was that StaticArrays did not target the true bottleneck of the model. After Conv2D was

optimized using `im2col` + GEMM, `MaxPool` was no longer the dominant cost. Therefore, the `StaticArrays` version was rejected.

This experiment is a useful counterexample: an optimization that is theoretically reasonable for small arrays may not improve the full application if it is applied outside the actual bottleneck.

XII. MACRO ANNOTATIONS

Several macro-based optimizations were also tested, including:

- `@inbounds`,
- `@views`,
- `@simd`,
- `@inline`,
- `@fastmath`.

The results were mixed. Some local annotations were kept where they were safe and did not affect correctness, but most macro-only experiments did not provide stable improvement. In several cases, they made short benchmarks slower.

Therefore, macro annotations should not be interpreted as the main optimization of the project. The main performance improvement came from the algorithmic redesign of `Conv2D` and from reducing unnecessary allocations.

XIII. COMPARISON WITH REFERENCE IMPLEMENTATIONS

A reference implementation was prepared in `PyTorch`. Additionally, a `Flux` implementation was used as a `Julia` framework reference.

All reference implementations used:

- the same CNN architecture,
- `FashionMNIST`,
- 3 epochs,
- SGD,
- learning rate 0.01,
- Dropout (0.4).

The final comparison is shown in Table VII.

TABLE VII
FINAL COMPARISON

Version	Time	Accuracy
KM1 MiniAD baseline	67.91 s	87.13%
Final KM2 MiniAD	28.75 s	87.59%
<code>PyTorch</code> reference	20.18 s	84.86%
<code>Flux</code> reference	36.11 s	86.66%

The optimized MiniAD implementation is approximately:

$$\frac{67.91}{28.75} \approx 2.36$$

times faster than the KM1 baseline.

Compared with `PyTorch`, the final MiniAD version is slower, but it reaches higher test accuracy in this specific run. The accuracy difference should not be interpreted as

a general superiority of MiniAD, because the result may depend on initialization, implementation details, and randomness. The key conclusion is that the optimized custom `Julia` implementation reached the required accuracy and training time constraints while remaining reasonably close to a mature deep learning framework.

XIV. DISCUSSION

The profiling results confirmed that forward and backward propagation remain the dominant costs. This means that future work should focus on:

- more efficient convolution kernels,
- reducing `im2col` memory overhead,
- tiled or fused convolution,
- GPU acceleration,
- better memory reuse in backward propagation.

The rejected experiments show that optimization must be measurement-driven. Several intuitive micro-optimizations were slower in practice. This was especially visible for macro annotations and the `StaticArrays` `MaxPool` experiment.

XV. LIMITATIONS AND FUTURE WORK

The results apply to one CNN architecture, `FashionMNIST`, CPU execution, SGD, batch size 10, and a 3-epoch training schedule. Absolute times may depend on hardware, `Julia` version, BLAS backend, and threading configuration. Future work could include GPU acceleration and tiled convolution to reduce `im2col` memory overhead.

XVI. CONCLUSIONS

The KM2 project optimized MiniAD for CNN training on `FashionMNIST`. The main improvement was replacing direct `Conv2D` loops with `im2col` + BLAS GEMM. This reduced 3-epoch training time from 67.91 s to 28.75 s while maintaining 87.59% test accuracy. Memory measurements showed that final training allocations were reduced below the KM1 baseline, and evaluation allocations decreased by about 90%. The final implementation passed all unit tests and satisfied the required accuracy and time constraints.

REFERENCES

- [1] `Julia` Documentation, <https://docs.julialang.org/>
- [2] BLAS Documentation, <https://www.netlib.org/blas/>
- [3] `PyTorch` Documentation, <https://pytorch.org/docs/stable/index.html>
- [4] `Flux.jl` Documentation, <https://fluxml.ai/Flux.jl/stable/>
- [5] Xiao, H., Rasul, K., and Vollgraf, R., *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, arXiv:1708.07747, 2017.
- [6] Baydin, A., Pearlmutter, B., Radul, A., and Siskind, J., *Automatic Differentiation in Machine Learning: a Survey*, *Journal of Machine Learning Research*, 18, 153:1–153:43, 2018.
- [7] Anderson, A., Vasudevan, A., Keane, C., and Gregg, D., *Low-memory GEMM-based convolution algorithms for deep neural networks*, arXiv:1709.03395, 2017.